



XIAMEN
UNIVERSITY

第二讲 MATLAB基础知识 (2)

温程璐 人工智能系



知识要点

- 2.1 数据类型
- 2.2 运算符与运算
- 2.3 字符串处理
- 2.4 数组
- 2.5 高维数组
- 2.6 非数与空数组
- 2.7 文件和数据的导入与导出



2.3 字符阵列

在MATLAB中，每个字符按16位ASCII码储存，这大大方便了在MATLAB中使用双字节内码字符集，如汉字系统。

一、字符与ASCII码之间的变换

利用double函数和char函数可实现在字符与其ASCII码之间进行变换。

```
A1 = [65 66; 67 68];  
A2 = 'abcd' ;  
C = char(A1,A2) %多数组转化为单一字符阵列  
C = 3x4 char array  
'AB '  
'CD '  
'abcd'
```

```
A = [77 65 84 76 65 66];  
C = char(A) %由整数数组到字符数组  
C = 'MATLAB'
```

```
A = "Pythagoras" %双引号输入，产生一个字符串  
C = char(A) %将字符串转换为字符向量  
C = 'Pythagoras' %单引号为字符
```



2.3 字符阵列

String array 字符串数组

- R2016b 版本开始使用字符串数组代替字符数组用于表达文字
- 字符串数组中存储了一组字符
- 各组字符的长度可以不同
- 字符串数组如只有一个元素，成为字符串标量
- 如果字符串数组中表示的是数字，可以用double函数转换为数值数组

字符串数组生成

- 使用双引号产生字符串，如：str = "Hello, world"
- 使用方括号将字符串组合起来，产生字符串数组，如：
str = ["Mercury", "Gemini", "Apollo"; "Skylab", "Skylab B", "ISS"]
str = 2x3 string
"Mercury" "Gemini" "Apollo"
"Skylab" "Skylab B" "ISS"



2.3 字符阵列

字符和字符串

- 字符数组和字符串数组用于存储 MATLAB 中的文本数据。
- **字符数组**是一个字符序列，就像数值数组是一个数字序列一样。它的一个典型用途是将短文本片段存储为**字符向量**，如 `c = 'Hello World'`。要将数据转换为字符数组，可使用 `char` 函数。
- **字符串数组**是文本片段的容器。字符串数组提供一组用于将文本处理为数据的函数。从 R2017a 开始，可以使用双引号创建字符串，例如 `str = "Greetings friend"`。要将数据转换为字符串数组，可使用 `string` 函数。



2.3 字符阵列

字符和字符串区别

1. 字符数组和字符串数组是不同的数据类型：char与string。因此，定义了不同的函数集
2. 占用内存不同：字符串数组是容器类，在内存中占用更多字节
 - `a = 'abc'`
 - `b = string('abc')`
 - `whos a b`
 - | Name | Size | Bytes | Class | Attributes |
|------|------|-------|--------|------------|
| a | 1x3 | 6 | char | |
| b | 1x1 | 132 | string | |
3. 元素访问方式：要表示不同长度的字符向量，必须使用cell个数组，例如`ch = { 'a', 'ab', 'abc' }`。使用字符串，可以直接创建：`str = [string( 'a' ), string( 'ab' ), string( 'abc' )]`。对于字符串数组中的元素索引使用大括号法：`str{3}(2) % == 'b'`
4. 不是总能互换



2.3 字符阵列

二、建立二维字符阵列

- 建立二维阵列时，应确保每行上的字符数相等
- 如果长度不等，应在其后补空格
- 可以利用**blanks(n)**函数来添加空格
- 利用**deblank**函数可以删除字符串末尾多余的空格

例如：

```
s1='welcome to xiamen university'
```

```
s2='you are welcome to my hometown'
```

```
s=[s1blanks(2);s2]
```



2.3 字符阵列

2.3.1 字符串单元阵列

字符串单元阵列中每个元素均为字符串，而且已经删除了末尾的空格。

1. 利用 `cellstr` 函数将字符阵列变换成字符串单元阵列

例如： `cell=cellstr(s)`

2. 利用 `char` 函数可以进行反变换

例如： `s=char(cell)`



2.3 字符阵列

2.3.2 字符串比较

比较字符串的三种方式：

1. 比较两个字符串或其部分是否相同
2. 比较两个字符串中个别字符是否相同
3. 可对字符串中的每个元素进行归类

一、比较字符串是否相同

- **strcmp**函数：用于比较字符串是否相同
- **strcmpi**函数：用于比较时忽略大小写
- **strncmp**函数：用于比较两个字符串的前n个字符是否相同
- **strncmpi**函数：比较时忽略大小写



2.3 字符阵列

2.3.2 字符串比较

一、比较字符串是否相同

- **strcmp**函数：用于比较字符串是否相同
- **strcmpi**函数：用于比较时忽略大小写
- **strncmp**函数：用于比较两个字符串的前n个字符是否相同
- **strncmpi**函数：比较时忽略大小写

例如：

s1= 'help' , s2= 'hello' , s3= 'Hello'

k1=strcmp(s1,s2) %比较两个串是否相同

则k1=0

k2=strcmpi(s2,s3) %比较时忽略大小写

则k2=1



2.3 字符阵列

2.3.2 字符串比较

一、比较字符串是否相同

- `strncmp`函数：比较两个字符串的前n个字符是否相同。如果二者相同，函数将返回 1 (true)，否则返回 0 (false)。如果两个文本段的内容一直到结尾都相同或前 n 个字符相同（以先出现者为准），则将这两个文本段视为相同。返回结果的数据类型为 logical。
- 可以比较字符串数组、字符向量和字符向量元胞数组的任何组合。
- `strncmpi`函数：比较时忽略大小写

```
s1 = 'Kansas City, KS' ;  
s2 = 'Kansas City, MO' ;  
tf = strncmp(s1,s2,11) %比前11，相同  
tf = logical 1
```

```
s1 = ["Jacques"; "Jean"; "Jeanne"; "Jean-Luc"; "Julie"];  
s2 = "Jean";  
tf = strncmp(s1,s2,4) %比较字符串数组，前4个  
tf = 5x1 logical array  
0  
1  
1  
1  
1  
0
```



2.3 字符阵列

2.3.2 字符串比较

二、比较字符是否相同
利用关系操作符。

三、英文字母的检测

isletter函数。isletter(A) 返回逻辑数组。如果 A 是字符数组或字符串标量，则当 A 中的某个字符是字母时，返回值对应的元素是逻辑值 1 (true)，否则是逻辑值 0 (false)。如果 A 不是字符数组或字符串标量，则返回逻辑值 0 (false)。

```
str = "123 Main St."
```

```
TF = isletter(str)
```

```
TF = 1x12 logical array
```

```
0 0 0 0 1 1 1 1 0 1 1 0
```



2.3 字符阵列

2.3.2 字符串比较

三、英文字母的检测

`isspace`函数。`isspace(A)` 返回逻辑数组。如果 `A` 是字符数组或字符串标量，则当 `A` 中的某个字符是空白字符时，返回值对应的元素为逻辑值 1 (`true`)；否则为逻辑值 0 (`false`)。`isspace` 可识别所有 Unicode® 空白字符。

如果 `A` 不是字符数组或字符串标量，`isspace` 将返回逻辑值 0 (`false`)。

```
chr = '123 Main St.'
```

```
TF = isspace(chr)
```

```
TF = 1x12 logical array
```

```
0 0 0 1 0 0 0 0 1 0 0 0
```



2.3 字符阵列

2.3.3 字符串搜索与取代

- findstr (查找某个字符串)
 - strmatch (字符串匹配)
 - strrep (修改字符串)
 - strtok (提取字符串的首部)
- …等函数可以完成字符串的搜索与取代



2.3 字符阵列

2.3.4 字符串与数值之间的变换

常用的有

- `int2str`(数值转换为字符)
- `num2str` (含有小数的数值转换为字符)
- `bin2dec` (二进制转换为十进制)
- `hex2dec` (十六进制转换为十进制)
- `base2dec` (三进制转换为十进制)
- `dec2base` (十进制转换为三进制)



2.3 字符阵列

2.3.5 其他字符与字符串函数

一、一般命令

1. char

功能：建立字符矩阵

格式： `s=char (x)`

2. double

功能：字符阵列变换成双精度数值

格式： `y=double(x)`

3. cellstr

功能：从字符阵列中建立字符串单元阵列

格式： `c=cellstr (s)`



2.3 字符阵列

2.3.5 其他字符与字符串函数

二、字符阵列测试

1. ischar

功能：检测到字符阵列时为逻辑真

格式：k=ischar (a)

三、字符串操作

1. strcat

功能：字符串连接

格式：t=strcat (s1, s2, s3....)

2. strvcats

功能：字符串的直接连接

格式：t=strvcats (s1, s2, s3....)



2.4 数组

2.4.1 数值数组的创建和寻访

1. 递增/减型一维数组的创建

(1) “冒号”生成法 $X=a:inc:b$

A 是数组第一个元素；inc是步长（默认为1）

最后一个元素为

$$\left\{ a + inc \cdot \left[\frac{b-a}{inc} \right] \right\}$$

(2) 线性（或对数）定点法

$X=\text{ linspace}(a,b,n)$ a,b为左右端点，产生线性等间隔1*n行数组，n为个数

$x=\text{ logspace}(a,b,n)$ a,b为左右端点，产生对数等间隔1*n行数组，n为个数



2.4 数组

2.4.1 数值数组的创建和寻访

2. 其他类型一维数组的创建

- (1) 逐个元素输入法
- (2) 运用MATLAB函数生成法

【例2.1-1】一维数组的常用创建方法举例。

```
a1=1:6
```

```
a2=0:pi/4:pi    % A 是数组第一个元素; inc是步长（默认为1）最后一个元素为
```

```
a3=1:-0.1:0
```

```
a1 =
```

```
1      2      3      4      5      6
```

```
a2 =
```

```
0      0.7854      1.5708      2.3562      3.1416
```

```
a3 =
```

```
Columns 1 through 8
```

```
1.0000      0.9000      0.8000      0.7000      0.6000      0.5000  
0.4000      0.3000
```

```
Columns 9 through 11
```

```
0.2000      0.1000      0
```



2.4 数组

2.其他类型一维数组的创建

- (1) 逐个元素输入法
- (2) 运用MATLAB函数生成法

```
b1=linspace(0,pi,4)      % a,b为左右端点，产生线性等间隔1*n行数组，n为个数
b2=logspace(0,3,4)      % a,b为左右端点，产生对数等间隔1*n行数组，n为个数
b1 =
      0      1.0472      2.0944      3.1416
b2 =
      1      10     100    1000
c1=[2  pi/2  sqrt(3)  3+5i]
c1 =
      2.0000      1.5708      1.7321      3.0000 + 5.0000i
rng default      %控制随机数生成
c2=rand(1,5)      %生成随机数
c2 =
      0.8147      0.9058      0.1270      0.9134      0.6324
```

[[补充]]

```
x1=(1:6)', x2=linspace(0,pi,4)'
y1=rand(5,1)
z1=[2; pi/2; sqrt(3); 3+5i]
```



2.4 数组

2.4.2 二维数组的创建

1 小规模数组的直接输入法

【例2.1-2】在MATLAB环境下，用下面三条指令创建二维数组c

```
a=2.7358; b=33/79;
```

```
c=[1, 2*a+i*b, b*sqrt(a); sin(pi/4), a+5*b, 3.5+i]
```

c =

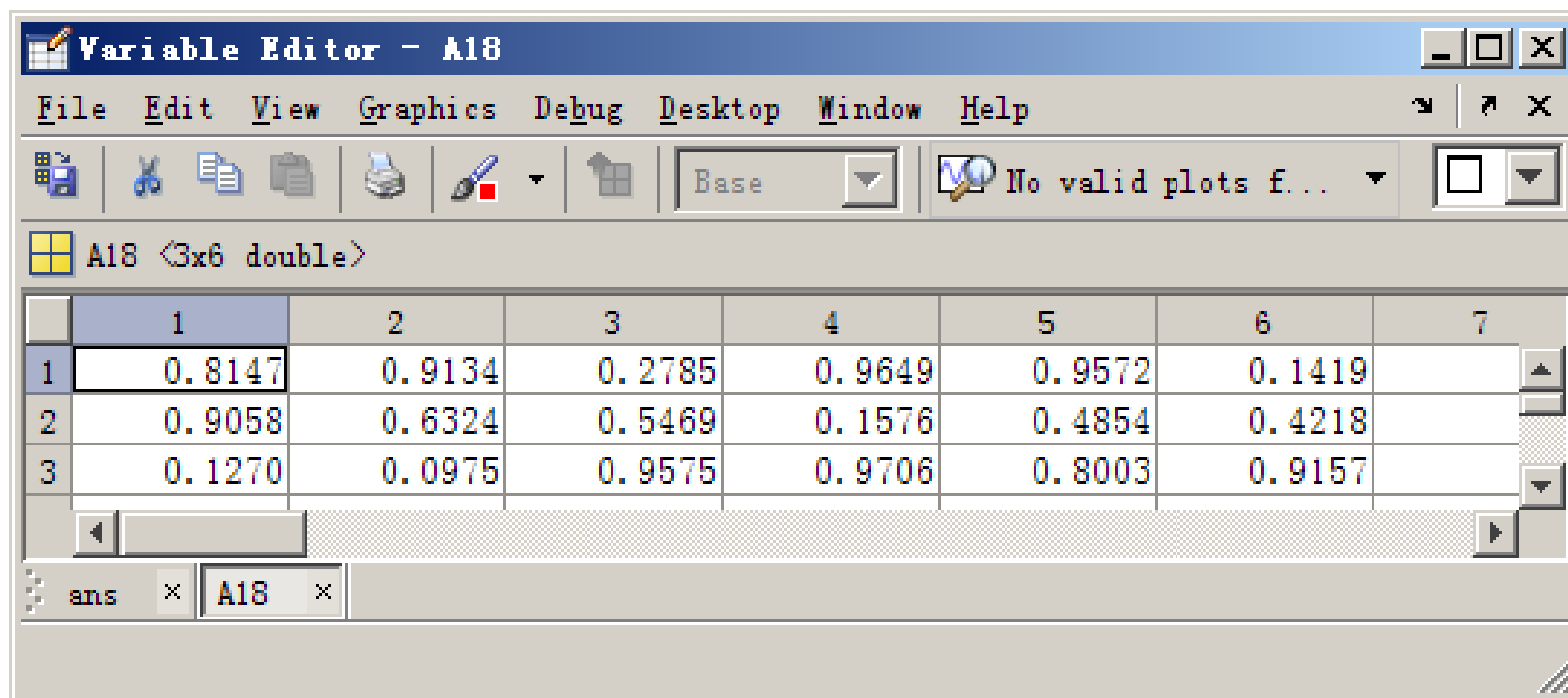
1.0000	5.4716 + 0.4177i	0.6909
0.7071	4.8244	3.5000 + 1.0000i



2.4 数组

2.4.2 二维数组的创建

2 中规模数组的数组编辑器创建法

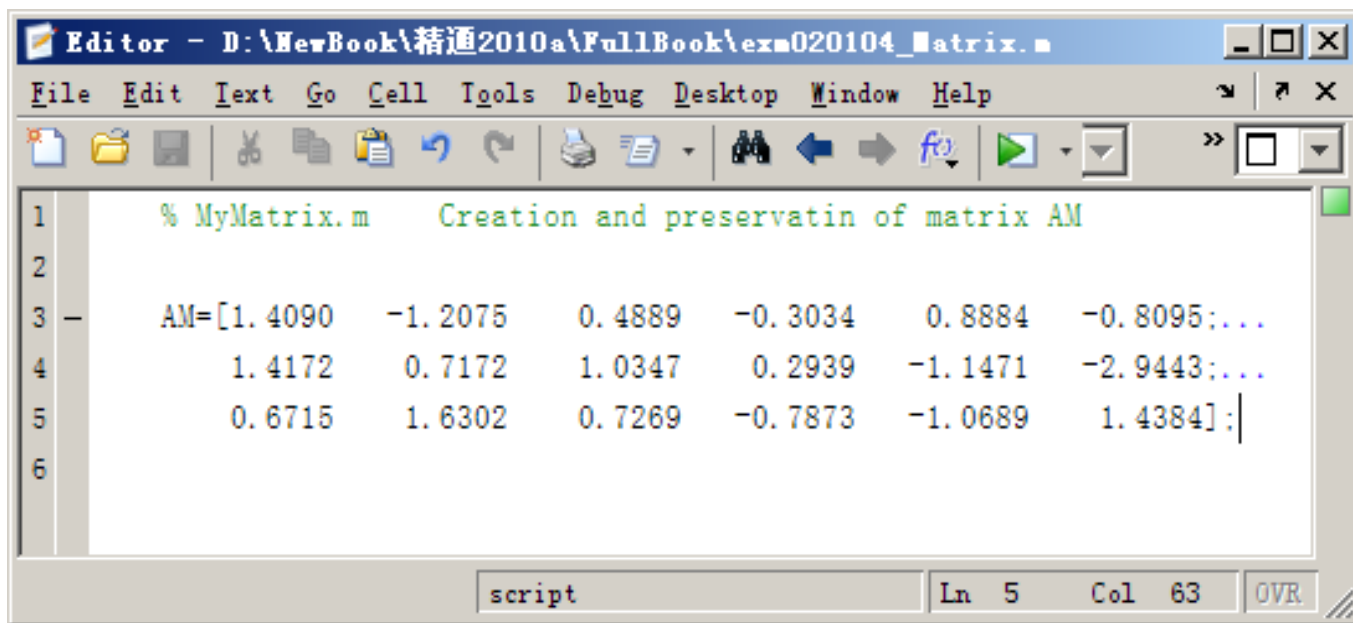


2.4 数组

2.4.2 二维数组的创建

3 中规模数组的M文件创建法

【例2.1-4】为数组AM，创建一个 exm020104_Matrix.m 文件。以后当需要AM数组时，只要运行exm020104_Matrix文件，就可在内存生成AM。



The screenshot shows a MATLAB Editor window titled "Editor - D:\NewBook\精通2010a\FullBook\exm020104_Matrix.m". The window contains a script file with the following content:

```
1 % MyMatrix.m      Creation and preservatin of matrix AM
2
3 -   AM=[1.4090    -1.2075    0.4889   -0.3034    0.8884   -0.8095;...
4       1.4172    0.7172    1.0347    0.2939   -1.1471   -2.9443;...
5       0.6715    1.6302    0.7269   -0.7873   -1.0689    1.4384];
6
```

The status bar at the bottom indicates "script", "Ln 5", "Col 63", and "OVR".



2.4 数组

2.4.2 二维数组的创建

4 利用MATLAB函数创建

【例2.1-5】利用MATLAB指令产生数组。

(1)

```
ao=ones(2,4)      %创建全1数组
az=zeros(2,5)     %创建全0数组
ae=eye(3)         %创建单位数组
am=magic(4)       %创建魔方数组
ad=diag(am)       %创建对角线数组
add=diag(diag(am)) %对角元阵数据
```

ao =

1	1	1	1
1	1	1	1

az =

0	0	0	0	0
0	0	0	0	0

ae =

1	0	0
0	1	0
0	0	1

am =

16	2	3	13
5	11	10	8
9	7	6	12
4	14	15	1

ad =

16
11
6
1

add =

16	0	0	0
0	11	0	0
0	0	6	0
0	0	0	1



2.4 数组

4 利用MATLAB函数创建

```
(2)
rng default          % 产生相同的随机数
Au=rand(1,5)         % 均匀分布的1x5随机数组
Ai=randi([-3,4],2,8)  %产生【-3, 4】中整数的均匀分布（2x8）随机数
As=randsrc(3,12,[-3,-1,1,3],1) % [-3,-1,1,3]字符集上产生(3x12)均布数组
Ap=randperm(8)        % 产生8个正整数随机排列
Au =
    0.8147    0.9058    0.1270    0.9134    0.6324
Ai =
   -3     1     4     4     0    -2     4     4
   -1     4    -2     4     3     0     3     2
As =
   -1    -1    -3     1    -3     1    -3     3     3    -3    -3     1
     1    -3    -1    -1     3    -1    -3    -1     3    -3    -1     1
   -3    -3    -1     1    -3     1     3     1    -3     3     3    -1
Ap =
     1     7     8     4     6     5     2     3
rng(0,'v5normal')
randn(2,6)           % 产生2x6的正态随机矩阵
ans =
   -0.4326    0.1253   -1.1465    1.1892    0.3273   -0.1867
   -1.6656    0.2877    1.1909   -0.0376    0.1746    0.7258
```



(3)

```
n=5;lambda=2;
```

```
A = gallery('jordbloc',n,lambda) %产生主对角元为2的5阶Jordan
```

```
A =
```

```
    2    1    0    0    0
    0    2    1    0    0
    0    0    2    1    0
    0    0    0    2    1
    0    0    0    0    2
```

```
rng(11,'v5normal') % v5.0 normal generator
```

```
n=6;
```

```
kappa=1e8;
```

```
mode=2;
```

```
B= gallery('randsvd',n,kappa,mode) % 产生的矩阵元素服从正态分布
```

```
Bsv=svd(B) % B矩阵的奇异值
```

```
Bc=cond(B) % B矩阵的条件数
```

```
B =
```

```
-0.2402 -0.6286 -0.6241 -0.1413 0.2258 -0.2410
-0.5761 0.2703 0.2092 -0.1420 -0.2454 -0.4657
0.5168 -0.1311 0.0244 -0.6882 -0.4403 -0.2138
0.5613 0.2022 -0.1260 0.2781 0.3097 -0.1772
-0.0744 0.0489 0.3518 -0.5518 0.7473 0.0709
-0.1044 -0.2899 0.1391 -0.0840 -0.2010 0.7394
```

```
Bsv =
```

```
1.0000 1.0000 1.0000 1.0000 1.0000 0.0000
```

```
Bc =
```

```
1.0000e+008
```



2.4.3 二维数组元素的标识和寻访

1. 数组的维数和大小

(1) 数组的维数 (Dimension)

`ndims (A)` 直接给出数组A的维数

(2) 数组的大小 (Size)

`size (A)` 各维的大小, `length(A)` 给出最大长度

【例2.1-6】数组的维数、大小和长度

```
clear
A=reshape(1:24,2,3,4); %把24个量变成4页，每页2*3个元素
dim_A=ndims(A)
size_A=size(A)
L_A=length(A)
dim_A =
     3
size_A =
     2     3     4
L_A =
     4
```



2.4.3 二维数组元素的标识和寻访

2. 数组的标识和寻访

【例2.1-7】 本例演示：数组元素及子数组的各种标识和寻访格式；冒号的使用；end的作用。

(1)

A=zeros(2,6) %创建2x6的全0数组

A(:)=1:12 %赋值由1到12

a8=A(8) %单个下标获得数组A的第8个元素

a311=A([3,11]) %单个下标获得数组A的第3和11号元素

A =

0	0	0	0	0	0
0	0	0	0	0	0

A =

1	3	5	7	9	11
2	4	6	8	10	12

a8 =

8

a311 =

3	11
---	----



2.4.3 二维数组元素的标识和寻访

(2)

```
A(3,7)=37
```

%双下标, 寻找第3行第7列

```
a13=A(:, [1,3])
```

%双下标, 寻找A的1和3列全部元素

```
aend=A([2,3], 4:end)
```

% 双下标, 获得A第4列到最后1列, 第2和3行

A =

1	3	5	7	9	11	0
2	4	6	8	10	12	0
0	0	0	0	0	0	37

a13 =

1	5
2	6
0	0

aend =

8	10	12	0
0	0	0	37

(3)

```
L=A<3
```

%逻辑结果, 经比较小于3为1, 大于3为0

```
A(L)=NaN
```

% 逻辑为1的记成NaN

L =

1	0	0	0	0	0	1
1	0	0	0	0	0	1
1	1	1	1	1	1	0

A =

NaN	3	5	7	9	11	NaN
NaN	4	6	8	10	12	NaN
NaN	NaN	NaN	NaN	NaN	NaN	37



2.4.3 二维数组元素的标识和寻访

3. 数组的扩充和收缩

【例 2.1-8】二维数组的扩充和收缩。

(1)

```
a=1:5;b=6:10;c=11:15;
```

```
a_b=[a, b]           %逗号联排
```

```
ab=[a; b; c]         %分号分行
```

```
a_b =
```

```
     1     2     3     4     5     6     7     8     9    10
```

```
ab =
```

```
     1     2     3     4     5
     6     7     8     9    10
    11    12    13    14    15
```

(2)

```
AB1=repmat(ab,[1,2]) %重复排列块, 扩展列
```

```
AB2=repmat(ab,[2,1]) %重复排列块, 扩展行
```

```
AB1 =
```

```
     1     2     3     4     5     1     2     3     4     5
     6     7     8     9    10     6     7     8     9    10
    11    12    13    14    15    11    12    13    14    15
```

```
AB2 =
```

```
     1     2     3     4     5
     6     7     8     9    10
    11    12    13    14    15
     1     2     3     4     5
     6     7     8     9    10
    11    12    13    14    15
```

【例 2.1-8】二维数组的扩充和收缩

(3)

```
AB2([2,3,5,6],:)=[] %删除某行
```

```
AB2(:,1:3)=[] %删除某些列
```

```
AB2 =
```

```
     1     2     3     4     5
     1     2     3     4     5
```

```
AB2 =
```

```
     4     5
     4     5
```



2.4.3 二维数组元素的标识和寻访

4. 数组的特殊操作

数组变形、翻转、平移和排序指令

【例 2.1-9】本例演示：reshape的数组变形功能；数组的翻转指令flipud, fliplr, flipdim，以及它们体现的矩阵变换；数组绕“左上元素”反时针旋转指令rot90；数组上下左右平移回绕指令circshift。

(1)

```
clear
```

```
a=1:24;           % 定义一个一维数组
```

```
A=reshape(a,3,8)   % 将数组变形为3行，8列
```

```
B=reshape(A,2,4,3) % 将数组变形为，2X4X3的多维数组，3个2X4
```

```
A =
```

```
1  4  7 10 13 16 19 22
2  5  8 11 14 17 20 23
3  6  9 12 15 18 21 24
```

```
B(:,:,1) =
```

```
1  3  5  7
2  4  6  8
```

```
B(:,:,2) =
```

```
9 11 13 15
10 12 14 16
```

```
B(:,:,3) =
```

```
17 19 21 23
18 20 22 24
```



2.4.3 二维数组元素的标识和寻访

4. 数组的特殊操作

数组变形、翻转、平移和排序指令

(3)

`A90=rot90(A)` %反时针90度

`A180=rot90(A,2)` %反转两次

`A90 =`

22	23	24
19	20	21
16	17	18
13	14	15
10	11	12
7	8	9
4	5	6
1	2	3

`A180 =`

24	21	18	15	12	9	6	3
23	20	17	14	11	8	5	2
22	19	16	13	10	7	4	1

(4)

`A`

`CR=circshift(A,1)` % 循环移动, 下移1行

`CL=circshift(A,[0,-1])` % 循环移动, 左移1列; 默认右移

`A =`

1	4	7	10	13	16	19	22
2	5	8	11	14	17	20	23
3	6	9	12	15	18	21	24

`CR =`

3	6	9	12	15	18	21	24
1	4	7	10	13	16	19	22
2	5	8	11	14	17	20	23

`CL =`

4	7	10	13	16	19	22	1
5	8	11	14	17	20	23	2
6	9	12	15	18	21	24	3



2.4.4 数组运算

- 基本运算：一般常用的加 (+)、减 (-)、乘 (*)、除 (/) 的数学运算符号，以及幂次运算 (^)，是在矩阵意义下进行的，单个数据的运算是一个特例
- 除加减运算外，如果要进行标量(矩阵元素)的运算必须加`.`，如`.*`，`./`
- 进行基本数学运算时，只需将运算式直接打入提示号 (>>) 之後，并按入Enter键即可

• 例如：

```
>> (5*2+1.3-0.8)*10/25  
  
ans =  
  
4.2000
```

- MATLAB会将运算结果直接存入一变量ans，代表MATLAB运算后的答案 (Answer)，并显示其数值于萤幕上,也可以将结果赋值给变量，例如：

```
>> x=5*10  
  
x =  
  
50
```



2.4.4 数组运算

- **MatLab基本运算**

- MATLAB可在同时执行数个命令，只要以逗号或分号将命令隔开：

- $x = \sin(\pi/3); y = x^2; z = y*10$

- $z =$

7.5000

- 若一个数学运算是太长，使用三个句点将其延伸到下一行：

- $z = 10*\sin(\pi/3)* \dots$

- $\sin(\pi/3);$

- 若要检查当前工作空间（Workspace）的变量，可键入whos 命令

```
>> whos
```

Name	Size	Bytes	Class
a	1x1	8	double array
b	3x3	72	double array

```
Grand total is 10 elements using 80 bytes
```



2.4.4 数组运算

- 数组运算和向量化编程（优点）

【例 2.2-1】欧姆定律： $r = \frac{u}{i}$ ，其中 r, u, i 分别是电阻（欧姆）、电压（伏特）、电流（安培）。验证实验：据电阻两端施加的电压，测量电阻中流过的电流，然后据测得的电压、电流计算平均电阻值。（测得的电压电流具体数据见下列程序）。

（1）非向量化编程方式

```
clear
vr=[0.89, 1.20, 3.09, 4.27, 3.62, 7.71, 8.99, 7.92, 9.70, 10.41];
ir=[0.028, 0.040, 0.100, 0.145, 0.118, 0.258, 0.299, 0.257, 0.308, 0.345];

L=length(vr);           %数组长度
for k=1:L
    r(k)=vr(k)/ir(k);    % 数组各元素顺序相除
end

sr=0;
for k=1:L
    sr=sr+r(k);          %加和
end
rm=sr/L                  % 求平均值
rm = 30.5247
```



2.4.4 数组运算

- 数组运算和向量化编程

【例 2.2-1】欧姆定律： $r = \frac{u}{i}$ ，其中 r, u, i 分别是电阻（欧姆）、电压（伏特）、电流（安培）。验证实验：据电阻两端施加的电压，测量电阻中流过的电流，然后据测得的电压、电流计算平均电阻值。（测得的电压电流具体数据见下列程序）。

(2) 向量化编程

```
Clear
vr=[0.89, 1.20, 3.09, 4.27, 3.62, 7.71, 8.99, 7.92, 9.70, 10.41];
ir=[0.028, 0.040, 0.100, 0.145, 0.118, 0.258, 0.299, 0.257, 0.308,
0.345];
r=vr./ir      %数组元素相处
rm=mean(r)    %数组元素求均值
r =
    Columns 1 through 8
    31.7857    30.0000    30.9000    29.4483    30.6780    29.8837
    30.0669    30.8171
    Columns 9 through 10
    31.4935    30.1739
rm =
    30.5247
```



• 数组特殊运算指令汇总

【例2.2-3】数组元素的“和”、“积”、“累和”、“累积”运算。

```
rng default
a=[ (1:5) ',randi(5,[5,3]),randn(5,2)]    %生成数组1-5, 5以内随机5x3大小, 正太分布随机
cs=cumsum(a)                               % 每列元素的累计和
s=sum(a)                                    % 每列全部元素和
cp=cumprod(a)                              % 每列全部累计积
p=prod(a)                                  % 每列全部元素积
```

a =

1.0000	5.0000	1.0000	1.0000	-0.2050	0.6715
2.0000	5.0000	2.0000	5.0000	-0.1241	-1.2075
3.0000	1.0000	3.0000	5.0000	1.4897	0.7172
4.0000	5.0000	5.0000	3.0000	1.4090	1.6302
5.0000	4.0000	5.0000	5.0000	1.4172	0.4889

cs =

1.0000	5.0000	1.0000	1.0000	-0.2050	0.6715
3.0000	10.0000	3.0000	6.0000	-0.3291	-0.5360
6.0000	11.0000	6.0000	11.0000	1.1606	0.1812
10.0000	16.0000	11.0000	14.0000	2.5696	1.8115
15.0000	20.0000	16.0000	19.0000	3.9868	2.3004

s =

15.0000	20.0000	16.0000	19.0000	3.9868	2.3004
---------	---------	---------	---------	--------	--------

cp =

1.0000	5.0000	1.0000	1.0000	-0.2050	0.6715
2.0000	25.0000	2.0000	5.0000	0.0254	-0.8108
6.0000	25.0000	6.0000	25.0000	0.0379	-0.5816
24.0000	125.0000	30.0000	75.0000	0.0534	-0.9481
120.0000	500.0000	150.0000	375.0000	0.0757	-0.4635

p =

120.0000	500.0000	150.0000	375.0000	0.0757	-0.4635
----------	----------	----------	----------	--------	---------



2.5 高维数组

2.5.1 高维数组的创建

- (1) 全下标元素赋值法
- (2) 若干同样大小低维数组组合
- (3) ones, zeros, rand, rand直接创建
- (4) cat, repmat, reshape等创建高维数组



2.5.1 高维数组的创建

- (1) 全下标元素赋值法
- (2) 若干同样大小低维数组组合
- (3) ones, zeros, rand, rand直接创建
- (4) cat, repmat, reshape等创建高维数组

【例2.3-1】“全下标”元素赋值方式创建高维数组演示。

(1)

A(2,4,2)=1 % 数组4*3, 两页, 赋值第二页最后一个元素

A(:, :, 1) =

0	0	0	0
0	0	0	0

A(:, :, 2) =

0	0	0	0
0	0	0	1

(2)

C=ones(2,3);

C(:, :, 3)=ones(2,3)*3 % 每页分别赋值

C(:, :, 1) =

1	1	1
1	1	1

C(:, :, 2) =

2	2	2
2	2	2

C(:, :, 3) =

3	3	3
3	3	3



2.5.2 高维数组的孤维删除

- Squeeze 删除A中孤维保持元素位置不变
- Shiftdim 删除A阵的前导孤维

(1)

```
A=reshape(1:24,[1,3,4,1,2]); % 创建5维数组, 尺寸为1x3x4x1x2
```

```
SA=size(A) %各维度大小
```

```
B=squeeze(A) % 删除单一维度, 剩下3*4*2
```

```
SA =
```

```
1 3 4 1 2
```

```
B(:, :, 1) =
```

```
1 4 7 10
```

```
2 5 8 11
```

```
3 6 9 12
```

```
B(:, :, 2) =
```

```
13 16 19 22
```

```
14 17 20 23
```

```
15 18 21 24
```

(2)

```
[Am,m]=shiftdim(A) % Am和A元素一样,  
去除单维数组
```

```
Am(:, :, 1, 1) =
```

```
1 4 7 10
```

```
2 5 8 11
```

```
3 6 9 12
```

```
Am(:, :, 1, 2) =
```

```
13 16 19 22
```

```
14 17 20 23
```

```
15 18 21 24
```

```
m =
```

```
1
```



2.5.3 高维数组的维度重排

- Permute 重排A的维度次序;
- Ipermute 逆操作

【例2.3-3】高维数组的维度重排。

(1)

`A=reshape(1:24,[2,4,3])` %2X4X3大小

`A(:, :, 1) =`

1	3	5	7
2	4	6	8

`A(:, :, 2) =`

9	11	13	15
10	12	14	16

`A(:, :, 3) =`

17	19	21	23
18	20	22	24

`DimOrder=[3,2,1];`

`B=permute(A,DimOrder)` %变成【3, 4, 2】

`AA=ipermute(B,DimOrder)` % 逆变换

`B(:, :, 1) =`

1	3	5	7
9	11	13	15
17	19	21	23

`B(:, :, 2) =`

2	4	6	8
10	12	14	16
18	20	22	24

`AA(:, :, 1) =`

1	3	5	7
2	4	6	8

`AA(:, :, 2) =`

9	11	13	15
10	12	14	16

`AA(:, :, 3) =`

17	19	21	23
18	20	22	24



2.6 非数与空数组

2.6.1 非数

(1) 非数的产生和性质

NaN非数，具有以下特性：

- NaN参与运算也是NaN，有传递性
- 非数没有“大小”概念

(2) 非数的功用有：

- 避免运算中断
- 真实记录结果
- 数据可视化的时候，用来裁剪图形

【例2.4-1】非数的产生和性质演示。

(1)

```
a=0/0,b=0*log(0),c=inf-inf
```

```
a =
```

```
NaN
```

```
b =
```

```
NaN
```

```
c =
```

```
NaN
```

(2)

```
0*a,sin(a)
```

```
ans =
```

```
NaN
```

```
ans =
```

```
NaN
```

(3)

```
class(a)
```

```
isnan(a) %是否为非数，逻辑判断
```

```
ans =
```

```
double
```

```
ans =1
```



2.6 非数与空数组

2.6.1 非数

【例2.4-2】非数元素的寻访。

(1)

```
rng default
```

```
R=rand(2,5);R(2,3)=NaN;R(1,5)=NaN
```

```
R =
```

```
0.8147 0.1270 0.6324 0.2785 NaN
0.9058 0.9134 NaN 0.5469 0.9649
```

(2)

```
LR=isnan(R) %是否为非数
```

```
LR =
```

```
0 0 0 0 1
0 0 1 0 0
```

(3)

```
si=find(LR); %定位非数的位置
[ri,ci]=ind2sub(size(R),si); %确定非数位置的
“单下标”标识
disp('非数位置的单下标标识')
disp(['第',int2str(si(1)), '和第',int2str(si(2)), '个
元素'])
disp(' ')
disp('非数位置的双下标标识')
disp(['第 ',mat2str([ri(1),ci(1)],2), ' 元素'])
disp(['第 ',mat2str([ri(2),ci(2)],2), ' 元素'])
非数位置的单下标标识
第6和第9个元素
非数位置的双下标标识
第 [2 3] 元素
第 [1 5] 元素
```



2.6 非数与空数组

2.6.1 非数

(4) find指令直接找“全下标”

```
[rj,cj]=find(LR);           %确定非数位置的“全下标”标识  
disp('非数位置的双下标标识')  
disp(['第 ',mat2str([rj(1),cj(1)],2),' 元素'])  
disp(['第 ',mat2str([rj(2),cj(2)],2),' 元素'])  
非数位置的双下标标识  
第 [2 3] 元素  
第 [1 5] 元素
```



2.6 非数与空数组

2.6.2 空数组

“空”数组是某维长度为0或若干维长度为0，则该就是“空”数组
空数组的功用

(1) 没有“空”数组参与运算时，计算结果中的“空”解释为“所得结果的含义”；

(2) 运用“空”数组对其他非空数组赋值，可以使数组变小，但不能改变数组的维数。

【例2.4-3】关于“空”数组的算例。

(1)

```
a=[]  
b=ones(2,0),c=zeros(2,0),d=eye(2,0)  
f=rand(2,3,0,4)  
a =  
    []  
b =  
    Empty matrix: 2-by-0  
c =  
    Empty matrix: 2-by-0  
d =  
    Empty matrix: 2-by-0  
f =  
    Empty array: 2-by-3-by-0-by-4
```



2.6 非数与空数组

2.6.2 空数组

```
(2)
class(a)
isnumeric(a)    %数组类吗
isempty(a)       %是否为“空”
ans =
double
ans =
     1
ans =
     1
which a
ndims(a)
size(a)
a is a variable.
ans =
     2
ans =
     0     0
```

```
(3)
A=reshape(-4:5,2,5)%-4到5, 变2x5
A =
    -4    -2     0     2     4
    -3    -1     1     3     5

A(:,[2,4])=[] %删除其中第2和4列

A =
    -4     0     4
    -3     1     5
```



2.7 文件和数据

2.7.1 文件和数据导入与导出

- 数据基本操作

- 文件的存储

1. 保存整个工作区
2. 保存工作区中的变量
3. 利用save命令（数据保存）

- 数据导入

MATLAB中导入数据通常由函数load实现

- 文件的打开

MATLAB中可以使用open命令打开各种格式的文件，MATLAB自动根据文件的扩展名选择相应的编辑器。



2.7 文件和数据

2.7.2 文本文件的读写

1. csvread、csvwrite

2. dlmread、dlmwrite

3. textread, textscan

函 数	功 能
csvread	读入以逗号分隔的数据
csvwrite	将数据写入文件，数据间以逗号分隔
dlmread	将以 ASCII 码分隔的数值数据读入到矩阵中
dlmwrite	将矩阵数据写入到文件中，以 ASCII 分隔
textread	从文本文件中读入数据，将结果分别保存
textscan	从文本文件中读入数据，将结果保存为单元数组



2.7 文件和数据

2.7.3 低级文件I/O

1. 标记

2. 宽度和精度指示

3. 转换字符

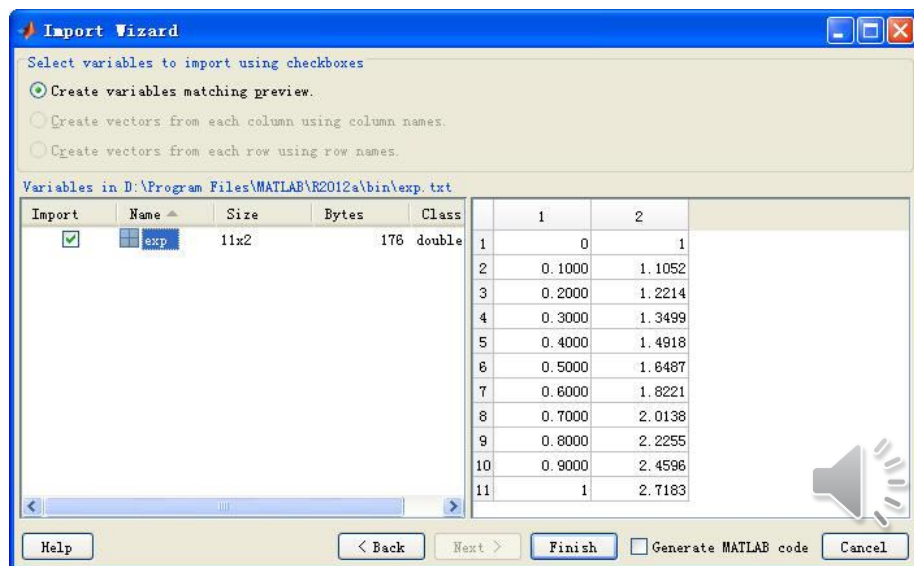
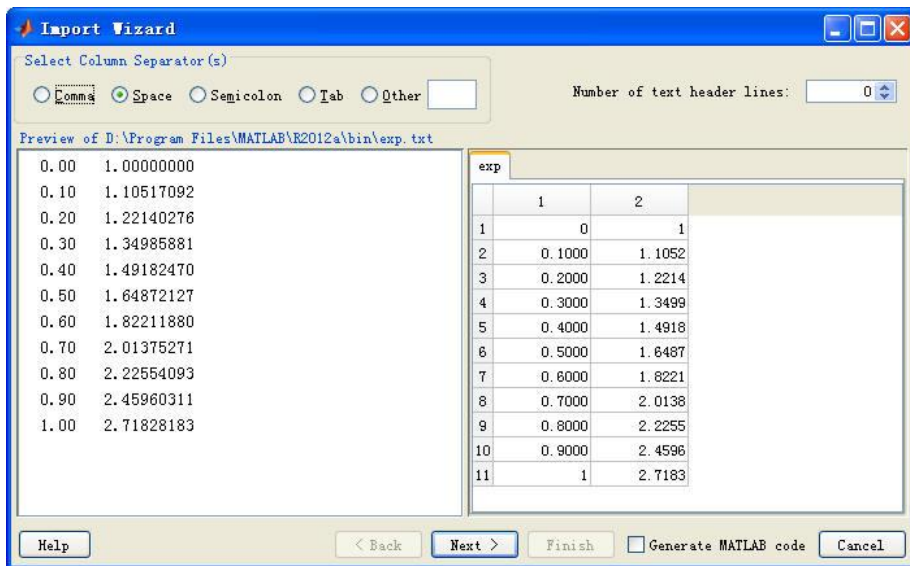
函 数	功 能
fclose	关闭打开的文件
feof	判断是否为文件结尾
ferror	文件输入输出中的错误查找
fgetl	读入一行，忽略换行符
fgets	读入一行，直到换行符
fopen	打开文件，或者获取打开文件的信息
fprintf	格式化输入数据到文件
fread	从文件中读取二进制数据
frewind	将文件的位置指针移至文件开头位置
fscanf	格式化读入
fseek	设置文件位置指针
ftell	文件位置指针
fwrite	向文件中写入数据



2.7 文件和数据

2.7.4 利用界面工具导入数据

- 单击工作区浏览器工具栏中的**Import Data**按钮，在弹出的对话框中选择待导入的文件，这里选择一个文本文件，其内容为逗号分隔的数字。
- 在该窗口中选择分隔符号，设置导入数据的起始行。在左侧窗口中显示的是文件中的内容，右侧窗口中是导入数据的预览。设置完成后，单击**Next**，进入下一界面。在该界面中可以设置导入方式，预览导入的变量。



本章结束，谢谢大家！

